ggfunction: Functions and Distributions in ggplot2

by Dusty Turner, David Kahle, Rodney X. Sturdivant, and James Otto

Abstract The ggfunction package provides ggplot2 layers for plotting mathematical functions, probability distributions, empirical distribution summaries, and censored-data summaries directly from function definitions or samples. Its design has three parts. First, a small dimensional taxonomy covers scalar functions, parametric curves, scalar fields, and vector fields. Second, probability layers display density, mass, distribution, survival, quantile, hazard, and cumulative-hazard functions, with support-aware tail, interval, and highest-density-region shading. Third, empirical layers compute ECDFs, empirical quantiles, PP/QQ/SP diagnostics, empirical mass functions, empirical cumulative hazards, Kaplan–Meier survival curves, and censored-data cumulative hazards. The package implements these displays as standard ggplot2 stats and geoms, so they compose with scales, coordinates, themes, facets, and annotations. This article describes the layer design, the main workflows, numerical conversion routes among distribution functions, validation checks, and limitations. DKW/Massart bands are finite-sample simultaneous bands for complete samples and fully specified continuous diagnostic nulls; fitted-null diagnostic bands are exploratory guides rather than calibrated composite-null acceptance regions. For censored data, the Kaplan–Meier layer uses an asymptotic simultaneous Greenwood/Nair band; the Nelson–Aalen layer uses a pointwise normal band.

1 Introduction

Many statistical graphics begin with a function rather than a data frame. An instructor shades the upper tail of a t distribution, a student compares an empirical CDF with a fitted model, or an analyst checks whether a censored sample is consistent with an exponential survival curve. In base R, these tasks are straightforward but often require hand-built grids, numerical integration, and ad hoc annotations. In **ggplot2** (Wickham, 2010, 2016), the built-in `stat_function()` covers only the univariate case $f : \mathbb{R} \to \mathbb{R}$ and leaves probability-specific calculations to the user.

ggfunction addresses this gap by making functions layer-level objects in **ggplot2**. Users supply an R function, optional parameters in `args`, and a domain. The layer evaluates the function, performs any requested statistical transformation, and returns ordinary **ggplot2** data for rendering. Figure 1 illustrates the motivating case: the user asks for a distributional tail probability, and the layer handles the quantile lookup, support-aware shading, and **ggplot2** display.

The package is available from CRAN and can be installed with `install.packages("ggfunction")`. Development versions and issue tracking are available from `https://github.com/dusty-turner/ggfunction`. The package depends on **ggplot2**; bivariate highest-density regions are delegated to **ggdensity** (Otto and Kahle, 2023), and vector-field and stream-field layers delegate to **ggvfields** (Turner et al., 2025).

The rest of the article is organized around package design and workflows rather than around a complete reference list. Section 2 describes the layer design and dimensional taxonomy. Section 3 presents three core workflows: mathematical functions, probability distributions, and empirical or censored data summaries. Section 4 summarizes implementation, numerical behavior, validation, and performance. Sections 5–7 discuss composability, related work, limitations, and future directions.

2 Design

2.1 Functions as layer-level objects

ggfunction does not change the grammar of graphics itself. It supplies new `ggplot2::layer()` constructors. Each constructor combines a `Stat` that evaluates a function or computes an empirical summary with a `Geom` that renders the returned data. This follows the **ggplot2** extension model: the result is a standard layer that can be added with `+`, trained by ordinary scales, clipped by coordinates, faceted, themed, and annotated.

The key interface choice is that functions are layer parameters, not aesthetics. This is deliberate. A function such as `dnorm` or a bivariate density is not a column in the plot data; it is the object the stat evaluates to create plot data. Additional function parameters are supplied through `args`, mirroring `ggplot2::stat_function()` and avoiding anonymous wrappers:

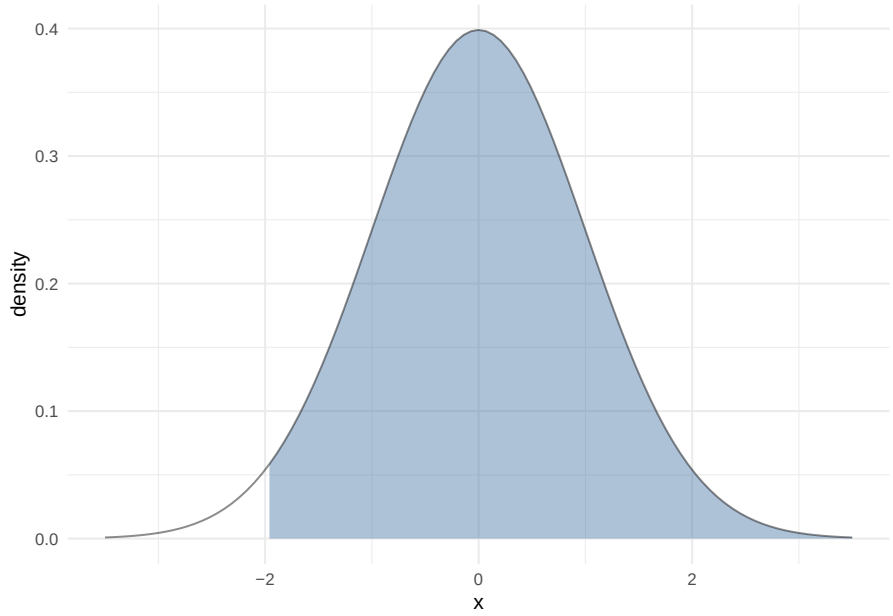

Figure 1: A motivating example: support-aware normal-density shading. The shaded upper tail is computed from the distribution, not by renormalizing over the displayed x range.

Table 1: Dimensional taxonomy for mathematical-function layers.

Signature	Layer	Display
$f : \mathbb{R} \to \mathbb{R}$	`geom_function_1d_1d()`	curve, optionally shaded
$\gamma : \mathbb{R} \to \mathbb{R}^2$	`geom_function_1d_2d()`	parametric path with parameter colour
$f : \mathbb{R}^2 \to \mathbb{R}$	`geom_function_2d_1d()`	raster, contours, or filled contours
$\mathbf{F} : \mathbb{R}^2 \to \mathbb{R}^2$	`geom_function_2d_2d()`	arrows or streamlines

```
ggplot() +
  geom_pdf(
    fun = dnorm,
    args = list(mean = 5, sd = 2),
    xlim = c(0, 10)
  )
```

This design keeps the common case short while preserving explicit control over the domain, grid resolution, and rendering choices.

2.2 Dimensional taxonomy

The mathematical-function family is organized by domain and codomain dimension. Table 1 gives the taxonomy and Figure 2 shows the corresponding displays.

The names encode the function signature. That convention is more verbose than `geom_function()`, but it makes the dimensional assumptions visible at the call site. Probability and empirical layers use statistical names instead: `geom_pdf()`, `geom_ecdf()`, `geom_ecdf_km()`, and so on.

3 Three workflows

3.1 Mathematical functions by dimension

For mathematical functions, the user's main decisions are the signature, the domain, and the visual encoding. A scalar function is evaluated on an equally spaced sequence over `xlim`. A parametric curve is evaluated over `tlim`; the returned length-two vector supplies the path coordinates. A scalar field is evaluated on an $n_x \times n_y$ grid and then handed to the corresponding **ggplot2** raster or contour

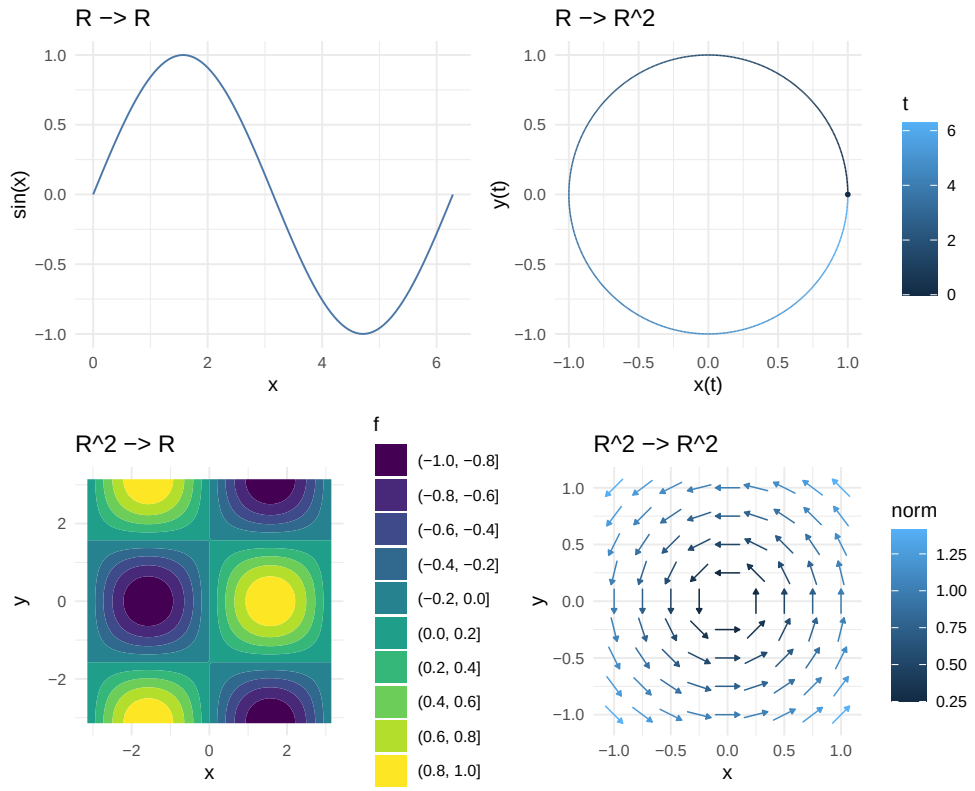

Figure 2: The four mathematical-function signatures supported by the ggfunction dimensional taxonomy: scalar functions, parametric curves, scalar fields, and vector fields.

machinery. A vector field is evaluated at seed points and then drawn by **ggvfields**; stream fields also delegate integration settings such as `method`, `max_it`, `T`, and `L`.

This workflow is useful when the data of interest are generated by a known mathematical rule. It also gives probability and diagnostic layers a common implementation pattern: create the grid, evaluate a function or summary, and return ordinary plot data.

The taxonomy is intentionally small. It covers the function signatures that can be displayed naturally in a two-dimensional plotting window without a separate three-dimensional graphics system. A scalar field, for example, can be displayed by mapping its output to colour or contour level while retaining the two input coordinates as the horizontal and vertical axes. A vector field can be displayed by drawing short vectors or streamlines at selected seed points. Higher-dimensional functions require additional choices–conditioning, projection, slicing, animation, or interactivity–that fall outside this package's current scope.

Two implementation details matter in this workflow. First, evaluation is grid-based. Increasing `n` improves smoothness and contour resolution, but it also increases computation, especially for two-dimensional grids. Second, `args` is spliced into the user's function at evaluation time. This lets a single function represent a parametric family without redefining closures:

```
lissajous <- function(t, a = 3, b = 2, delta = pi / 2) {
  c(sin(a * t + delta), sin(b * t))
}

ggplot() +
  geom_function_1d_2d(
    fun = lissajous,
    tlim = c(0, 2 * pi),
    args = list(a = 5, b = 4)
  )
```

For scalar and vector fields, the same convention means that model parameters, physical constants, or tuning values can remain explicit in the plotting call. This is especially helpful in teaching settings,

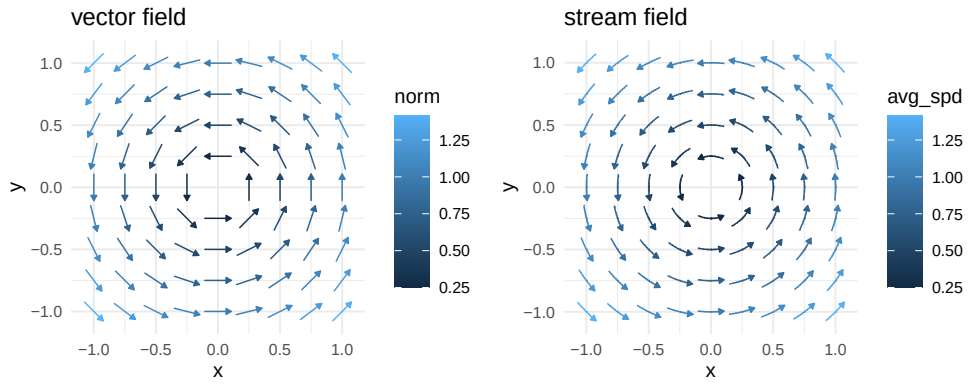

Figure 3: The same rotation field rendered as local arrows and as streamlines. ggfunction supplies the function interface and delegates vector-field rendering to ggvfields.

where the relationship between parameters and shape is often the point of the graphic. It also helps avoid subtle reproducibility problems: the plot source records both the mathematical function and the parameter values used to evaluate it.

Vector fields can be rendered either as local arrows or as streamlines. Both views start from the same vector-valued function, but they answer different visual questions. Arrows emphasize local direction and magnitude at selected grid points; streamlines emphasize integral curves of the field. In **ggfunction**, both displays are delegated to **ggvfields**, as shown in Figure 3.

3.2 Probability distributions

The probability workflow starts from one characterization of a distribution and asks for another visual representation. For example, users may have a density but want a CDF, a CDF but want a quantile function, or a hazard function but want a density. Native inputs are preferred when available. When a native input is absent, **ggfunction** can derive the needed function through the routes shown in Tables 2 and 3.

The continuous one-dimensional layers use the standard identities among distribution functions. If f is a density and F is the CDF, then

$$F(x) = \int_{-\infty}^{x} f(t)\,dt, \qquad S(x) = 1 - F(x), \qquad Q(p) = F^{-1}(p).$$

For survival and reliability displays, the hazard and cumulative hazard are

$$h(x) = \frac{f(x)}{S(x)}, \qquad H(x) = -\log S(x), \qquad S(x) = \exp\{-H(x)\}.$$

The package does not rederive these relationships for the reader each time a geom is used. Instead, it encodes them in conversion helpers that are shared by the distribution layers. This keeps the user-facing API small while making the route explicit in documentation and validation tests.

Discrete distributions use analogous operations on support points. A PMF is accumulated by summation rather than integration, and a quantile function is a generalized inverse over the ordered support. Because discrete supports can be finite, infinite, integer-valued, or explicitly supplied, the support argument is more than a plotting convenience. It defines the order in which cumulative mass is accumulated, determines where step functions jump, and controls how probability shading is resolved when multiple support points share the same mass.

The bivariate probability layers have a narrower scope. `geom_pdf_2d()` draws theoretical bivariate densities as HDRs or rasters, and `geom_pmf_2d()` draws bivariate mass functions on a lattice. The package does not estimate bivariate densities from samples; that task belongs to **ggdensity**. It also

Table 2: Continuous distribution-function inputs and conversion routes. Native inputs are used directly; derived routes may require numerical integration, finite differences, interpolation, or root finding.

Target	Native	Other sources	Route
PDF f	`fun`	F, S, Q, or h	finite diff., interpolation, or hS
CDF F	`fun`	f, S, Q, or h	integration or identity
survival S	`fun`	F, f, Q, or h	$1 - F$
quantile Q	`fun`	F, f, S, or h	root-find F
hazard h	`fun`	f, F, S, or Q	f/S; $f + F$ accepted
cum. hazard H	`fun`	h, F, S, f, or Q	$-\log S$ or integrate h

Table 3: Discrete distribution-function inputs and conversion routes. Computations are performed over the declared support.

Target	Native	Other sources	Route
PMF	`fun`	CDF or survival	support differences
CDF	`fun`	PMF or survival	cumulative sums
survival	`fun`	PMF or CDF	$1 - F$ on support
quantile	`fun`	PMF or CDF	generalized inverse

does not provide arbitrary three-dimensional rendering of density surfaces. The goal is to display probability content in the same two-dimensional grammar used by the rest of **ggplot2**.

A single source can therefore drive several views of the same distribution. Figure 4 starts from the gamma density and then asks **ggfunction** to draw related functions. Two of the panels use native functions, while the CDF and survival panels demonstrate PDF-derived routes. In a manuscript or analysis script, this makes the distributional relationship explicit in the plotting code.

The distinction between support and display window is important. The `xlim` argument controls what is drawn. The `support` argument controls distributional calculations: normalization checks, PDF-to-CDF integration, CDF-to-quantile inversion, and probability shading. Thus a 97.5% normal upper-tail request is interpreted with respect to the full normal distribution even if the visible window is narrower. Figure 5 shows lower-tail, central-interval, and two-tail requests.

This separation is easy to miss because ordinary plotting code often treats the x-axis limits as the whole domain. For probability displays, that would change the meaning of shaded area. Suppose a density has support $(-\infty, \infty)$ and the user displays only $[-2, 2]$. If a lower-tail request with $p = 0.95$ were normalized over the visible window, the shaded boundary would mark 95% of the mass inside $[-2, 2]$, not the 95th percentile of the distribution. That is rarely what a statistical diagram intends. In **ggfunction**, quantiles, tail probabilities, and central intervals are computed on the declared support; the coordinate system then determines which part of the result is visible.

The same distinction matters for bounded supports. A beta density should be integrated over $[0, 1]$, not over the displayed portion of that interval. A Weibull or exponential hazard should accumulate from time zero unless the user specifies another origin. A custom distribution with support on a nonstandard finite interval should declare that interval so numerical integration and root-finding do not waste effort outside the distribution's domain or return misleading boundary behavior. The support argument is therefore a statistical parameter of the layer, not a cosmetic axis setting.

Display windows still have an important role. They let the user focus on the region where a comparison is readable, crop extreme tails, or align several plots with a common x-axis. The recommended pattern is to compute on the distributional support and use the plotting window only for visual emphasis. For bivariate HDRs, the analogous pattern is to compute over `hdr_xlim/hdr_ylim` and use `coord_cartesian()` if a narrower view is desired. This is the same principle in two dimensions: probability content belongs to the computation domain, while readability belongs to the display domain.

Highest density regions (HDRs) are threshold-based regions of largest density containing a requested probability. In one dimension, **ggfunction** computes an approximate threshold on a grid over `hdr_xlim`. In two dimensions, `geom_pdf_2d()` adapts the package's vector-argument density interface to **ggdensity**'s `geom_hdr_fun()` and `geom_hdr_lines_fun()` interfaces. These bivariate HDRs are therefore gridded approximations whose accuracy depends on the computational domain and grid resolution. For a narrower view after computing on a wider domain, users should compute over the larger `hdr_xlim/hdr_ylim` and use `coord_cartesian()` for display.

The bivariate PMF layer uses the same vector-argument convention as the bivariate PDF layer, but

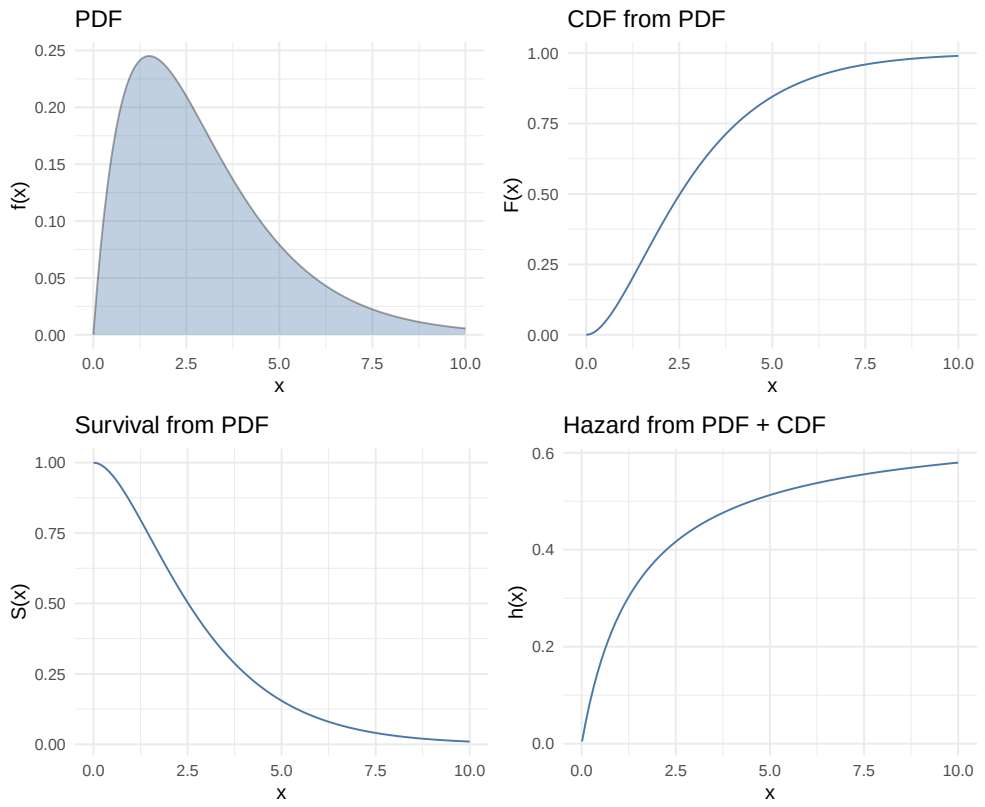

Figure 4: One gamma distribution viewed through four related functions. The PDF panel uses a native density; the CDF and survival panels derive from the density; the hazard panel uses the documented PDF + CDF bundle.

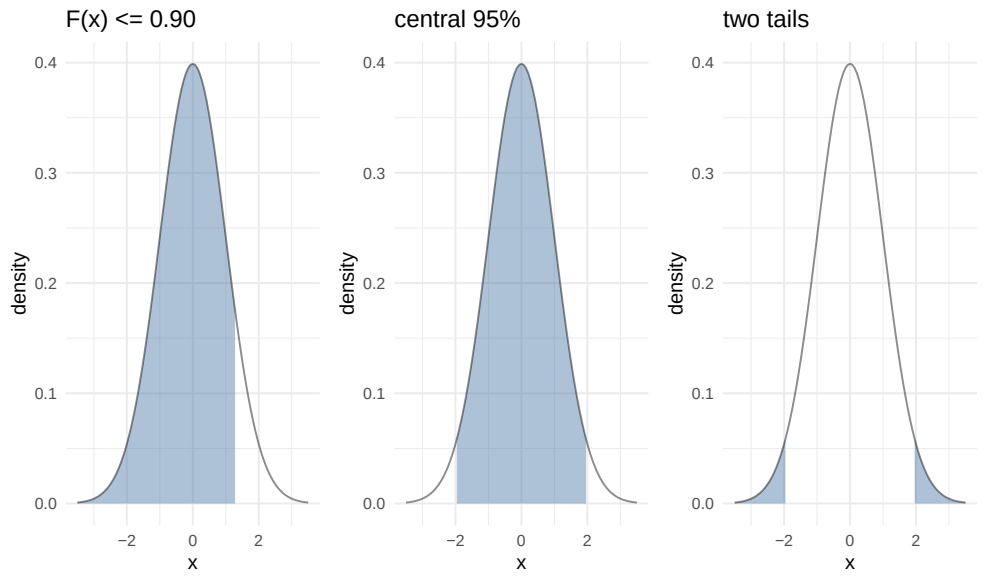

Figure 5: Three probability-shading requests for the standard normal distribution: a lower-tail probability, a central interval, and two tails outside a central interval.

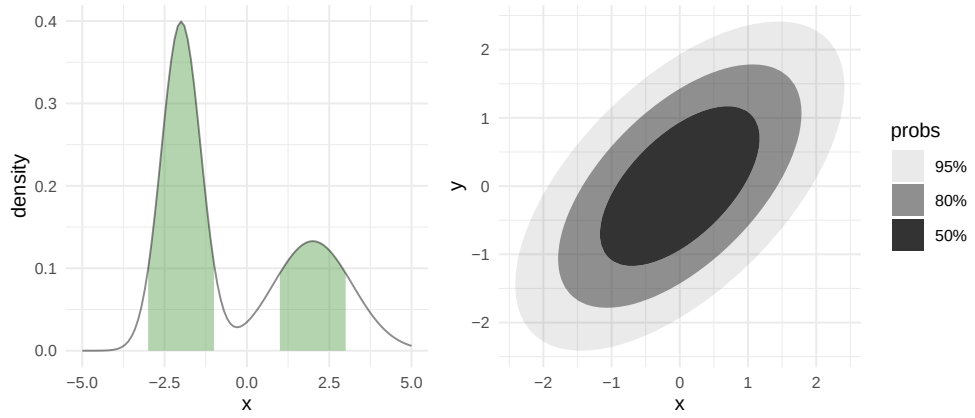

Figure 6: Approximate highest density regions. Left: a disconnected 80% grid-based HDR for a bimodal univariate density. Right: bivariate normal HDRs delegated to ggdensity.

evaluates on a lattice. Figure 7 shows the same independent product-binomial mass function in point and tile modes. Point mode maps probability to area, while tile mode maps probability to fill.

Discrete distributions follow the same ideas on a finite or integer support. The PMF layer uses lollipops, discrete CDF/quantile/survival layers use step functions, and discrete HDRs include all support points tied at an HDR cutoff. Figure 8 uses a deliberately nonconsecutive support to show why the support itself is part of the statistical object: cumulative sums and inverse lookups must be performed on the declared support, not on all integers between the minimum and maximum. For examples involving ties and bivariate mass functions, see the probability and numerical-conversion vignettes.

3.3 Empirical distributions and censored summaries

The empirical workflow compares sample summaries with distributional structure. For complete samples, `geom_ecdf()` and `geom_eqf()` draw empirical distribution and quantile functions. Their confidence bands use the Dvoretzky–Kiefer–Wolfowitz inequality with Massart's sharp constant (Massart, 1990). For a fully specified distribution F, the ECDF band

$$\hat{F}_n(x) \pm \sqrt{\frac{\log(2/\alpha)}{2n}}$$

has simultaneous coverage at least $1 - \alpha$, clipped to $[0, 1]$. Grouped ECDFs receive group-specific bands, as in Figure 9. The empirical quantile band is obtained by inverting these CDF limits. This inversion is visually useful but produces wide or partially uninformative tail regions when $p \pm \varepsilon$ reaches 0 or 1, which is a property of the DKW bound rather than a plotting artifact.

The previous version of this manuscript included a long simulation study of DKW coverage. That material is better suited to a validation vignette, where the number of simulations, distributions, and sample sizes can be varied without interrupting the package-paper narrative. The article therefore states the finite-sample guarantee and shows one representative workflow; the package vignette `coverage-simulation` contains the longer simulation record.

The same complete-data sample can be viewed on the cumulative-hazard scale. This is useful when the theoretical hazard has a simpler visual form than the CDF. For exponential data, the cumulative hazard is linear, so departures from linearity are easy to see. Figure 10 overlays a complete sample empirical cumulative hazard with the theoretical target.

Diagnostic layers use the same probability-scale argument. `geom_ppplot()` compares theoretical and empirical probabilities, `geom_qqplot()` compares theoretical and sample quantiles, and `geom_spplot()` applies Michael's arcsine-square-root stabilization (Michael, 1983) to the PP plot. When the null distribution is fully specified, the DKW/Massart band is a finite-sample simultaneous

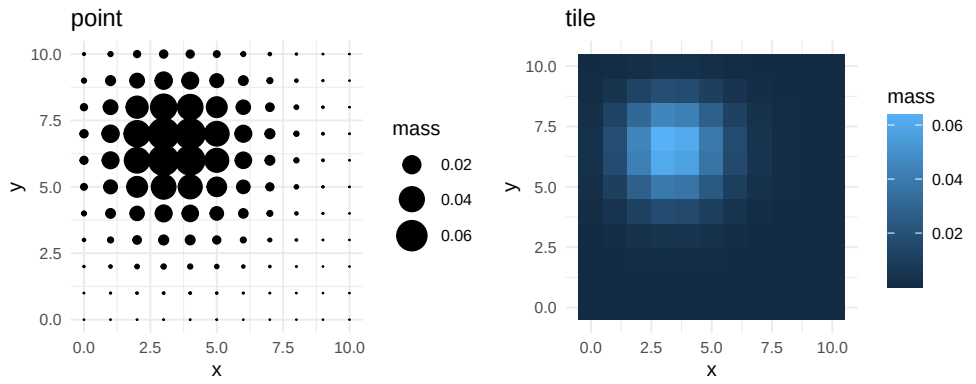

Figure 7: A bivariate probability mass function displayed as a balloon plot and as a tile heatmap.

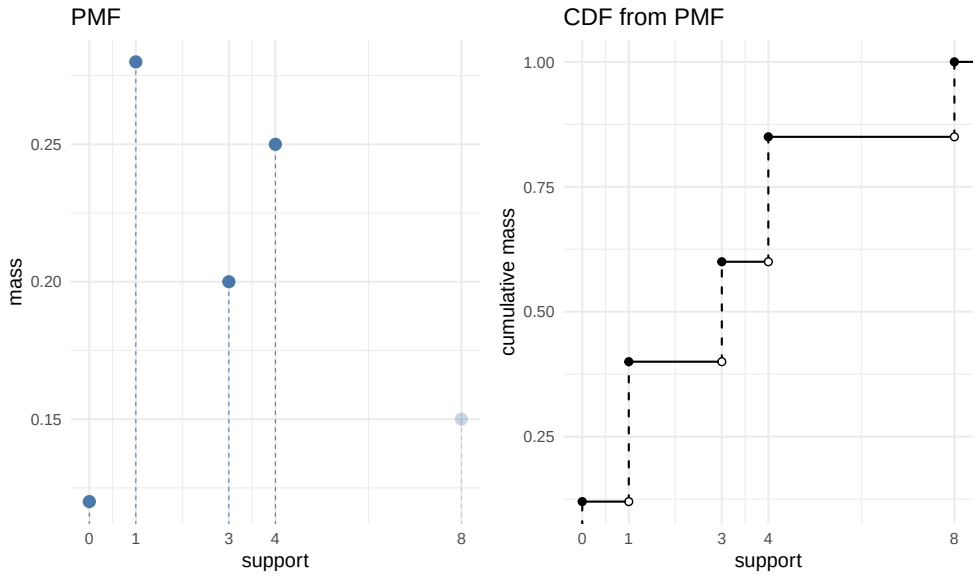

Figure 8: A discrete distribution on a nonconsecutive support. The PMF layer shades the lower cumulative region; the discrete CDF is derived from the same PMF by cumulative summation over the declared support.

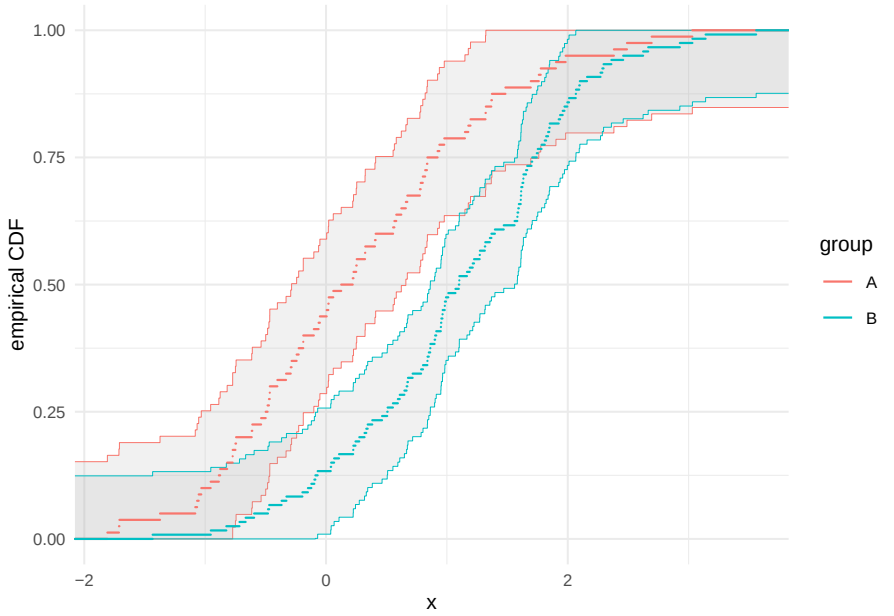

Figure 9: Grouped ECDFs with DKW/Massart confidence bands. Each group receives a separate band computed from its own sample size.

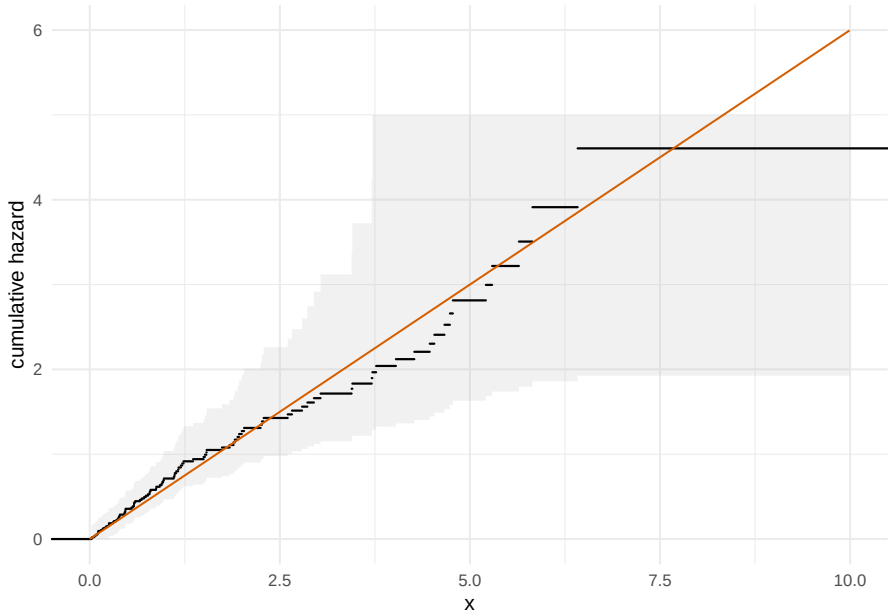

Figure 10: Complete-data empirical cumulative hazard with a transformed DKW band and the theoretical cumulative hazard for an exponential model.

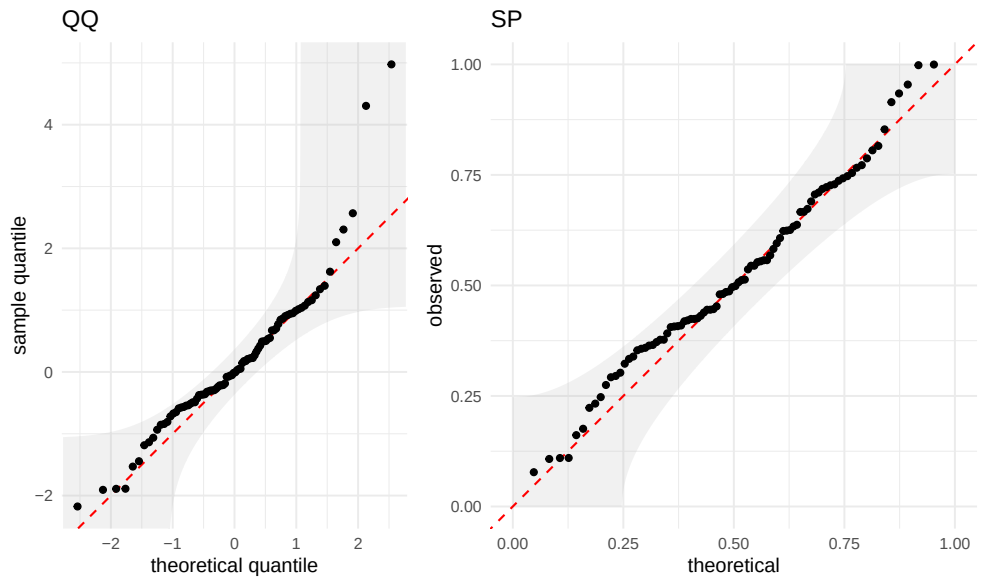

Figure 11: QQ and stabilized probability plots for one sample against a fully specified standard normal null. Fitted-null diagnostics use the same visual band but require separate calibration for formal testing.

band for the probability integral transform. When parameters are estimated from the same data being plotted, that argument no longer gives the stated finite-sample simultaneous calibration. This distinction is important in practice. In teaching, it is common to compare a sample with `pnorm` after estimating mean and standard deviation from the same data. **ggfunction** will draw the requested visual band, but it does not perform a Lilliefors or other composite-null adjustment. The plotted band can still be a useful exploratory diagnostic; users who need a calibrated composite-null goodness-of-fit procedure should use a method designed for that setting.

For right-censored data, `geom_ecdf_km()` computes the Kaplan–Meier product-limit estimator (Kaplan and Meier, 1958) from observed times and an event indicator. It uses risk sets and displays censoring marks by default. Its confidence band uses Greenwood's variance estimator (Greenwood, 1926) and the equal-precision construction of Nair (1984), which is simultaneous and asymptotic. The companion `geom_echf_na()` layer computes the Nelson–Aalen cumulative hazard estimator (Nelson, 1972; Aalen, 1978) and uses the implemented pointwise normal confidence band. Figure 12 shows the two estimators side by side. The package also includes `geom_echf()` for complete, uncensored samples. That layer applies the monotone transform $H(x) = -\log\{1 - F(x)\}$ to the ECDF and to the DKW band. For censored observations, however, ECDF-based summaries are no longer appropriate; the risk-set calculations in the Kaplan–Meier and Nelson–Aalen estimators are essential.

At each distinct event time t_j, let d_j denote the number of events and n_j the number at risk immediately before t_j. The Kaplan–Meier estimator displayed by `geom_ecdf_km()` is

$$\hat{S}(t) = \prod_{t_j \le t} \left(1 - \frac{d_j}{n_j}\right),$$

and the Nelson–Aalen estimator displayed by `geom_echf_na()` is

$$\hat{H}(t) = \sum_{t_j \le t} \frac{d_j}{n_j}.$$

The package computes both from the same internal risk-set table. This shared tabulation keeps event counts, censoring counts, risk sets, and variance estimates consistent between the two censored-data layers.

The confidence bands intentionally differ. The Kaplan–Meier band uses Greenwood's variance estimate and an equal-precision critical value following Nair's construction. The displayed limits are additive on the survival scale, $\hat{S}(t) \pm c_{\mathrm{EP}} \mathrm{se}\{\hat{S}(t)\}$, and are clipped to $[0, 1]$. The critical value is computed from Greenwood-scale endpoints at the first and last valid event times, so the band is a

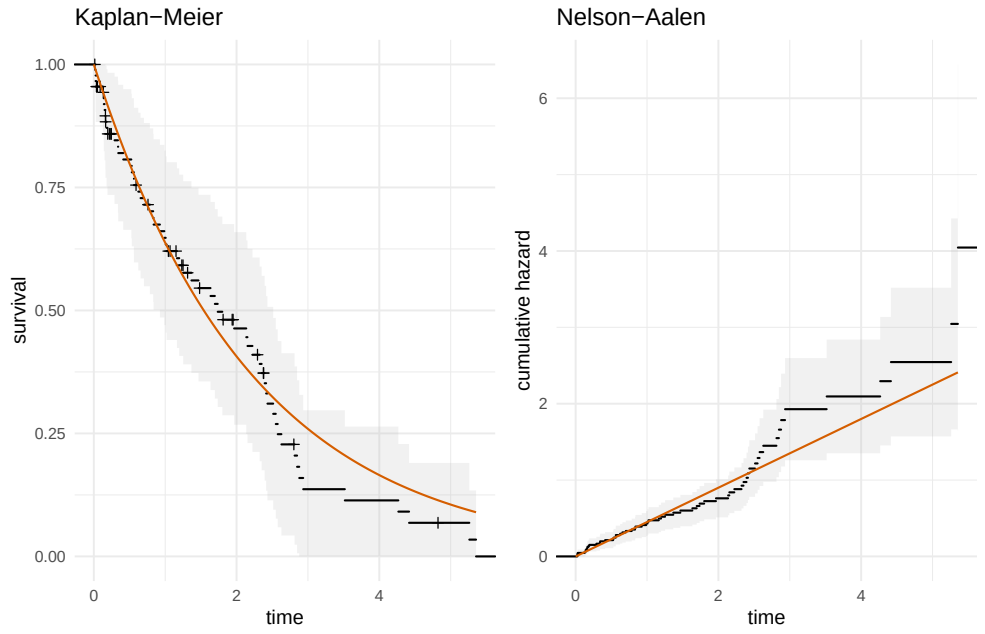

Figure 12: Censored-data summaries from the same simulated sample. Left: Kaplan–Meier survival with censor marks and a simultaneous Greenwood/Nair band. Right: Nelson–Aalen cumulative hazard with a pointwise normal band.

simultaneous large-sample band over the displayed event-time range, not an exact finite-sample band. The Nelson–Aalen band uses the Nelson variance estimate

$$\widehat{\mathrm{Var}}\{\hat{H}(t)\} = \sum_{t_j \le t} \frac{d_j}{n_j^2}$$

and a normal critical value at each time point. Its intervals are symmetric on the cumulative-hazard scale before the lower limit is clipped at zero. It is therefore pointwise, not simultaneous. The abstract and examples use this distinction deliberately.

4 Implementation, numerical behavior, and performance

4.1 ggplot2 stat-geom architecture

Each user-facing layer constructs one or more ggplot2 layers. The stat component validates inputs, evaluates functions or tabulates empirical summaries, and returns a data frame with computed variables such as `x`, `y`, `ymin`, `ymax`, or `probs`. The geom component draws those data using existing ggplot2 infrastructure when possible: paths and ribbons for curves and bands, rasters and contours for scalar fields, and step-like geoms for discrete and empirical functions.

The two main delegated components are explicit. Vector and stream fields are wrappers around ggvfields vector-field and stream-field layers. Bivariate PDF HDR displays are thin wrappers around ggdensity's functional HDR geoms; the raster mode for `geom_pdf_2d()` uses the scalar-field machinery instead. This delegation keeps **ggfunction** focused on function interfaces, probability semantics, and validation.

The layer constructors also provide a predictable data contract. Most users do not inspect the built data, but doing so is useful for tests, extensions, and debugging. Table 4 summarizes representative computed variables. These are not new aesthetics that users must supply; they are values created by the stat and then consumed by the geom.

This contract is also how the package is tested. Unit tests build plots with `ggplot_build()`, inspect the computed data frames, and compare band limits, support handling, route accuracy, and error messages with expected values. The tests therefore exercise the same data that a downstream extension would see.

Table 4: Representative computed variables returned by ggfunction stats.

Family	Variables	Geom
curves	x, y	path/area
fields	x, y, z	raster/contour
continuous prob.	x, y, shade vars	area/path/ribbon
discrete prob.	x, y, mass/HDR	lollipop/step
empirical	x, y, band limits	step/ribbon
censored	x, y, band/censor	step/ribbon/marks
diagnostics	x, y, order/probs	points/ribbon/line

4.2 Numerical conversions

The conversion code is intentionally conservative. Native functions are used directly. Derived functions are tagged internally as approximate, and their accuracy depends on smoothness, support, tail behavior, grid resolution, and whether the supplied function is numerically stable. The main conversion methods are:

- PDF to CDF: numerical integration over the declared support.
- CDF to PDF: central finite differences with support-aware endpoint handling.
- CDF to quantile: bracketing and `stats::uniroot()`.
- Quantile to CDF: monotone interpolation on a probability grid.
- Survival to CDF or cumulative hazard: algebraic transformations.
- Hazard to cumulative hazard: numerical integration from the hazard origin.
- Hazard to PDF: $f(x) = h(x)\exp\{-H(x)\}$ after deriving H.
- Discrete conversions: cumulative sums and generalized inverse lookup.

These routes are meant to support plotting, diagnostics, and teaching examples, not to replace distribution-specific numerical libraries. The numerical CDF derived from a smooth bounded density can be extremely accurate on a moderate grid. The same default settings may be inadequate for a density with sharp spikes near a boundary or for a heavy-tailed distribution whose visually interesting mass lies outside the displayed window. The API therefore exposes the most important controls–the declared support, the display window, the grid resolution, and the hazard integration origin–instead of hiding all numerical choices behind a single plotting call.

For hazards, the lower integration endpoint deserves special mention. If the support has a finite lower endpoint and `hf_lower` is left at its default of `-Inf`, the package uses the lower support endpoint. This makes common nonnegative lifetime distributions behave naturally when users write `support = c(0, Inf)`. If neither the support nor the hazard origin is finite, integrating a hazard can be ill-posed; the layer then warns or fails rather than silently drawing a misleading curve.

Tables 5 and 6 report representative local validation results from the accuracy and benchmark scripts in `inst/benchmarks/`. These scripts are manual validation aids rather than CRAN tests. They install the local package, compare fixed-grid function values, or time `ggplot_build()` for representative plots. The saved outputs used here are in `inst/benchmarks/results/` and record the run date, platform, R version, package versions, grid details, and benchmark iteration counts.

The reported run (2026-06-28 22:03:56 CDT) was on Darwin 25.5.0 arm64 using R 4.6.0 (2026-04-24), **ggfunction** 0.1.0, **ggplot2** 4.0.3, **ggdensity** 1.0.1, and **ggvfields** 1.0.0. The benchmark script used **bench** 1.1.4. Accuracy grids used 401 x-values for the normal, exponential, Weibull, and beta checks, 199 probabilities for quantile checks, and the 21-point binomial support.

The tables should be read as reproducibility checks, not universal guarantees. Heavy tails, discontinuities, bounded supports, singular hazards, and nearly flat CDF regions are harder cases. Users can improve behavior by supplying the native distribution function, declaring finite `support`, increasing `n`, or setting a finite `hf_lower` for hazards whose origin is known.

4.3 Performance and scaling

For native one-dimensional functions, build time is dominated by evaluating the function at `n` grid points and by **ggplot2** layer construction, so the cost scales roughly linearly in `n`. Scalar fields and bivariate density displays evaluate an $n_x \times n_y$ grid, so their computational cost scales like n^2. Conversions involving integration or root-finding are more expensive than native evaluations because they call numerical routines at many grid points. Stream fields depend additionally on ODE settings and the behavior of the vector field; rendering time can dominate for dense contours, HDRs, and streamlines.

Table 5: Representative numerical conversion accuracy from saved validation output. Errors are maximum absolute differences on the fixed grids described in the text.

Route	Distribution	Grid	Max error	Note
pdf->cdf	normal	401	5.82e-07	–
cdf->pdf	normal	401	1.29e-11	finite differences
cdf->qf	normal	199	3.49e-09	adaptive uniroot
hf->pdf	exponential	401	0.00e+00	support lower endpoint 0
hf->pdf	weibull	401	5.90e-08	support lower endpoint 0
pdf->cdf	beta(2,5)	401	3.33e-16	finite support
pmf->cdf	binomial(20,0.35)	21	5.55e-16	–
pmf->qf	binomial(20,0.35)	199	0.00e+00	–

Table 6: Representative local performance checks from saved benchmark output. Timings are medians over 10 iterations of plot construction, not full device rendering.

Case	Domain	Median	Iterations
geom_pdf(fun = dnorm)	101 x values on [-4, 4]	29.1ms	10
geom_pdf(cdf_fun = pnorm)	101 x values on [-4, 4]	28.1ms	10
geom_cdf(pdf_fun = dnorm)	101 x values on [-4, 4]	28.5ms	10
geom_function_2d_1d()	80 x 80 grid on [-pi, pi]^2	60.1ms	10
geom_pdf_2d(), ggdensity HDR	60 x 60 grid on [-3, 3]^2	140.6ms	10

4.4 Validation and warnings

The package validates common failure modes before plotting. Densities are checked for approximate normalization over their declared support, PMFs are checked for non-negative finite mass and approximate summation, CDF-like inputs are checked for monotonicity where the route requires it, and support/window diagnostics warn when an HDR computation domain omits substantial probability mass. Invalid source combinations are rejected. Most distribution geoms require exactly one source function, but hazard layers also accept the documented bundle `pdf_fun` + `cdf_fun`, which avoids deriving either component numerically.

The validation policy aims to catch mistakes that are easy to make in plotting code. Examples include using the displayed window as if it were the full support, supplying an unnamed `args` list, drawing a density that integrates far from one, evaluating a CDF-derived PDF outside the support, or asking for a quantile route without finite bracketing information. When the package can continue safely, it warns and returns missing values for failed points. When the requested route is logically ambiguous, it errors before building the plot.

The checks are deliberately lightweight enough for interactive plotting. They do not prove that an arbitrary user-supplied function is a valid distribution function. In particular, they cannot rescue a discontinuous CDF from finite differencing artifacts, identify every singular hazard, or choose a universally appropriate HDR grid. The documentation and numerical-conversion vignette therefore emphasize native distribution functions and explicit support declarations whenever accuracy matters.

The package also includes supporting infrastructure for submission and maintenance. Roxygen documentation records the accepted source combinations, approximation routes, support/window distinction, and warning behavior for each user-facing layer. Vignettes move longer teaching examples and validation stories out of the article: probability geoms, empirical geoms, numerical conversions, survival analysis, and DKW coverage simulation each have a focused home. Manual benchmark scripts in `inst/benchmarks/` provide reproducible accuracy and timing checks without running during `R CMD check`. This separation keeps CRAN checks fast while preserving evidence for manuscript claims.

5 Composability and limitations

Because **ggfunction** returns standard ggplot2 layers, its outputs compose with scales, coordinates, themes, facets, and annotations. A theoretical curve can be overlaid on an ECDF, a survival curve can be added to a Kaplan–Meier estimate, and a scalar field can be recolored with ordinary fill scales. This composability is the main practical benefit of implementing functions inside the ggplot2 layer system.

Composability also makes the package useful in documents and teaching material. A probability

layer can sit next to a simulation layer in a **patchwork** layout, use the same theme as the surrounding analysis, or inherit a coordinate system that zooms the display without changing the distributional calculation. The support/window distinction is especially important here: a plot can compute probability content over the full support while using `coord_cartesian()` to focus on the region where the visual comparison matters.

The same design also defines the limits. Functions are layer parameters, not aesthetics, so faceting over a data-frame column of function objects is not supported in the ordinary grammar-of-graphics sense. Numerical conversions are approximations and can fail for difficult distributions. HDRs are grid-based, so their thresholds depend on the computational domain and grid resolution. Two-dimensional grids can become expensive at high resolution. Censored-data bands have the validity described in the implementation: Kaplan–Meier uses the implemented simultaneous EP Greenwood/Nair band, while Nelson–Aalen uses the implemented pointwise normal band.

There are also statistical limits that no plotting interface can remove. Probability shading is only as meaningful as the distribution supplied to the layer. A diagnostic band for a fully specified null model has a different interpretation from the same band drawn after fitting parameters. A gridded HDR computed on too narrow a domain may have the wrong threshold. A survival curve with heavy censoring in the tail can have a visually wide band because the risk set is small. The package tries to make these situations visible, but the user remains responsible for choosing a model and interpreting the plot in context.

6 Related work

ggfunction is closest to tools that bridge statistical or mathematical objects and graphics. The compact summaries below describe the closest **ggplot2** extensions and adjacent packages. The intent is complementarity rather than replacement: several packages solve different problems better than **ggfunction** does.

Package	Focus	Relationship
`ggplot2::stat_function()`	$\mathbb{R} \to \mathbb{R}$	baseline curve layer
ggdist	uncertainty	broader displays
ggdensity	biv. HDRs	HDR engine used here
ggvfields	fields	delegated rendering

Package	Focus	Relationship
mosaic	teaching	broader pedagogy
metR	weather fields	geophysical displays
hdrcde	HDR/CDE	HDR objects

ggplot2 (Wickham, 2010, 2016) and Wilkinson's grammar (Wilkinson, 2005) provide the conceptual and software foundation. **ggdist** (Kay, 2024) is broader for uncertainty visualization. **ggdensity** (Otto and Kahle, 2023) supplies the bivariate HDR machinery used here. **ggvfields** (Turner et al., 2025) supplies vector-field and stream-field infrastructure. **mosaic** (Pruim et al., 2017), **metR** (Campitelli, 2024), and **hdrcde** (Hyndman et al., 2024) cover adjacent pedagogical, meteorological, and HDR-focused use cases.

The package boundaries are therefore partly philosophical. **ggdist** starts from distributions and uncertainty summaries and provides a rich visual vocabulary for communicating posterior draws, intervals, slabs, and dots. **ggfunction** starts from ordinary R functions and asks how they should be evaluated inside a **ggplot2** layer. **ggdensity** focuses on bivariate density estimation and probability-calibrated contours; **ggfunction** uses that machinery when the user supplies a theoretical bivariate density. **ggvfields** focuses on vector-field displays and integration; **ggfunction** exposes those displays through the dimensional taxonomy so scalar and vector-valued functions share a naming scheme.

This scope keeps the package lightweight. It does not define distribution objects, fit models, estimate bivariate densities from samples, or implement a general symbolic mathematics system. Instead it provides a bridge from function definitions and empirical summaries to ordinary **ggplot2** layers. That bridge is useful precisely because it leaves the rest of the graphics system unchanged.

7 Discussion and future directions

ggfunction makes a narrow commitment: take a mathematical or statistical function, evaluate it safely enough for plotting, and return a standard **ggplot2** layer. That commitment covers common teaching and diagnostic tasks without asking users to build grids, integrate densities, or hand-code confidence ribbons.

Future work could add better support for collections of functions, richer interfaces for distribution objects, more user controls for numerical conversion tolerances, conditional distributions and HDRs, additional censored-data bands, and higher-level wrappers for common teaching workflows. The benchmark and accuracy scripts added during package hardening provide a starting point for tracking these changes over time.

The most natural next step is support for function collections. Many classroom and applied examples compare a family of functions–normal densities with different scales, hazard functions under several parameter settings, or solution curves from different initial conditions. Today these can be overlaid by adding multiple layers manually. A future interface could make such collections more data-like while respecting the fact that functions are not ordinary aesthetics.

Another direction is more explicit numerical control. The defaults are chosen for readable interactive graphics, but some users will want tighter integration tolerances, adaptive grids, or route-specific diagnostics. Exposing those controls without turning simple plots into numerical-analysis exercises is a design challenge. The validation scripts introduced here give the package a baseline against which such changes can be checked.

8 Acknowledgments

James Otto contributed early drafts of the CDF and ECDF-related stats and geoms. Rodney Sturdivant contributed early design feedback. The authors used OpenAI Codex and Anthropic Claude as programming and drafting assistants during package development and manuscript revision; the authors reviewed and are responsible for the package code, examples, and manuscript claims.

References

O. Aalen. Nonparametric inference for a family of counting processes. *The Annals of Statistics*, 6(4): 701–726, 1978. doi: 10.1214/aos/1176344247. [p10]

E. Campitelli. *metR: Tools for Easier Analysis of Meteorological Fields*, 2024. URL https://CRAN.R-project.org/package=metR. R package. [p14]

M. Greenwood. *A Report on the Natural Duration of Cancer*. Number 33 in Reports on Public Health and Medical Subjects. His Majesty's Stationery Office, London, 1926. [p10]

R. J. Hyndman, J. Einbeck, and M. P. Wand. *hdrcde: Highest Density Regions and Conditional Density Estimation*, 2024. URL https://CRAN.R-project.org/package=hdrcde. R package. [p14]

E. L. Kaplan and P. Meier. Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282):457–481, 1958. doi: 10.1080/01621459.1958.10501452. [p10]

M. Kay. ggdist: Visualizations of distributions and uncertainty in the grammar of graphics. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):983–993, 2024. doi: 10.1109/TVCG.2023.3327195. [p14]

P. Massart. The tight constant in the Dvoretzky–Kiefer–Wolfowitz inequality. *The Annals of Probability*, 18(3):1269–1283, 1990. doi: 10.1214/aop/1176990746. [p7]

J. R. Michael. The stabilized probability plot. *Biometrika*, 70(1):11–17, 1983. doi: 10.1093/biomet/70.1.11. URL https://www.jstor.org/stable/2335939. [p7]

V. N. Nair. Confidence bands for survival functions with censored data: A comparative study. *Technometrics*, 26(3):265–275, 1984. doi: 10.1080/00401706.1984.10487964. [p10]

W. Nelson. Theory and applications of hazard plotting for censored failure data. *Technometrics*, 14(4): 945–966, 1972. doi: 10.1080/00401706.1972.10488991. [p10]

J. Otto and D. Kahle. ggdensity: Improved bivariate density visualization in R. *The R Journal*, 15(2): 220–236, 2023. doi: 10.32614/RJ-2023-048. [p1, 14]

R. Pruim, D. T. Kaplan, and N. J. Horton. The mosaic package: Helping students to "think with data" using R. *The R Journal*, 9(1):77–102, 2017. doi: 10.32614/RJ-2017-024. [p14]

D. Turner, D. Kahle, and R. X. Sturdivant. *ggvfields: Vector Field Visualizations with ggplot2*, 2025. URL https://CRAN.R-project.org/package=ggvfields. R package version 1.0.0. [p1, 14]

H. Wickham. A layered grammar of graphics. *Journal of Computational and Graphical Statistics*, 19(1): 3–28, 2010. doi: 10.1198/jcgs.2009.07098. [p1, 14]

H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2nd edition, 2016. ISBN 978-3-319-24277-4. URL https://ggplot2-book.org/. [p1, 14]

L. Wilkinson. *The Grammar of Graphics*. Springer-Verlag New York, 2nd edition, 2005. doi: 10.1007/0-387-28695-0. [p14]

Dusty Turner
United States Military Academy
West Point, NY
dusty.s.turner@gmail.com

David Kahle
Baylor University
Waco, TX
ORCiD: 0000-0002-9999-1558
david@kahle.io

Rodney X. Sturdivant
Baylor University
Waco, TX
rodney_sturdivant@baylor.edu

James Otto
Alcon Inc.
Fort Worth, TX
jamesotto852@gmail.com